SWE Assessment 2 Report Revision Pack
Prepared as a Word-formatted revision document for the Unity/C# project report.

Section 1.2 Why OOP Suits This Project

This project is not simply a game that happens to use classes. It is a real-time software system composed of many concurrent objects whose internal state changes at different rates. The player controller updates every frame, weapons update on input and cooldown events, AI agents react to sensing and navigation, and the arena director advances progression only when floor conditions are satisfied. An object-oriented design is appropriate because each runtime entity owns persistent state and exposes a limited interface to other systems.

The main benefit is encapsulation. PlayerController owns momentum, grounded state, dash charges, grapple state, and movement timers. Those variables are not edited directly by unrelated scripts. Instead, other systems interact through methods such as damage, interaction, or state-query calls. This reduces accidental coupling, which is important because movement is the core mechanic and is sensitive to small logic errors.

The second benefit is polymorphism through interfaces. The combat system does not need to branch on concrete target type before applying damage. Gun resolves a hit and then calls TakeDamage(float) on an IDamageable. The same call path works for the player, enemies, and target objects. The advantage is not only cleaner syntax. It means the weapon code depends on a stable contract rather than on the details of every target class, which lowers coupling and reduces repeated conditional logic.

The grapple system uses a second independent interface, IGrappleMassTarget, rather than forcing all grappleable objects into the damage interface. This is a better design because can be damaged and responds to grapple forces are separate responsibilities. BasicEnemyAI can implement both interfaces, while another object could implement only one of them. That separation is an example of interface segregation because each contract exposes only the behaviour required by the calling system.

Inheritance is used where shared behaviour is structural rather than incidental. Interactable is an abstract base class because all interactable world objects need a shared prompt message, focus feedback, interaction range, and a mandatory OnInteract(PlayerController) method. Similarly, PostProcessor defines a common Process(WFCGenerator3D) contract for generation passes that rewrite or repair the generated arena after collapse. In both cases, inheritance is used for a narrow and well-defined family of objects rather than for unrelated systems.

The project also depends heavily on composition, which is the normal Unity pattern. A BasicEnemyAI GameObject can include a NavMeshAgent, renderers, colliders, and audio or visual helpers without putting all behaviour into one parent class hierarchy. This is an advantage over deep inheritance because movement, combat, sensing, UI, and generation can evolve with less structural rigidity. In practice, the design uses inheritance for shared role contracts, interfaces for cross-system communication, and composition for runtime assembly.

Finally, OOP improves maintainability at the current project scale. The repository currently contains 68 C# scripts and about 35,407 lines of C# under Assets/Scripts. In a procedural design, the amount of shared mutable state and target-specific branching would become difficult to test and reason about. The current design is still imperfect because major classes such as PlayerController, Gun, BasicEnemyAI, and FINALArenaGenerator are too large, but the object model is still substantially more manageable than a monolithic loop-based program would be.

Section 1.4 Procedural Compared with OOP

The difference between procedural and object-oriented design in this project is most obvious in the way state and behaviour are distributed.

In a procedural version, player movement, enemy behaviour, and arena progression would

usually be implemented through a central update routine plus arrays, flags, and target-type conditionals. For example, the firing logic would need to test whether a hit object was an enemy, the player, or a destructible target, and then call different code paths for each one. As more content was added, the main routine would grow with more branches and more shared state. The implemented version instead pushes state and behaviour into the objects that own them.

Area	Procedural version	OOP version in this project
Enemy state	Health, position, aggro, and attack timers stored in arrays or global structures.	Each BasicEnemyAI instance stores its own health, navigation state, cooldowns, and sensing state.
Taking damage	Main loop checks target type and edits a matching variable or array slot.	Gun resolves a hit, obtains IDamageable, and calls TakeDamage(float).
Grapple response	Central code branches on enemy, crate, heavy object, or invalid target.	Grapple code queries IGrappleMassTarget and responds according to GrappleMassClass and ApplyGrapplePull(...).
Interaction	A long input branch checks doors, shops, terminals, and rewards separately.	PlayerController focuses an Interactable and calls OnInteract(player).
Arena repair	One generation script handles collapse, repair, decoration, and special-case fixes in one pass.	PostProcessor subclasses apply separate repair or population passes after generation.
Floor progression	Central flags decide if exits, shops, rewards, or bosses should activate.	CybergrindArenaDirector owns floor state and advances progression when terminals, enemies, and rewards are resolved.

The most important technical advantage is reduced coupling. Gun does not contain enemy-movement logic. BasicEnemyAI does not contain HUD logic. CybergrindShopStation does not regenerate the arena directly. Each class owns a constrained responsibility and communicates with neighbouring systems through a narrower contract.

This approach also changes how new content is added. In a procedural design, a new interactable or generation pass usually means inserting more logic into a central routine. In the current design, a new interactable can inherit from Interactable, implement OnInteract, and reuse the player focus and interaction pipeline. A new generation pass can inherit from PostProcessor and be inserted into the existing post-generation sequence. The extension point is therefore structural rather than ad hoc.

The main trade-off is that OOP does not automatically produce small or simple code. Several classes in this project have become large because they still accumulate too many responsibilities. That is a design debt issue, not a failure of the object-oriented model itself. The benefit is that the code already has identifiable seams for later refactoring, such as extracting movement state from PlayerController, weapon abilities from Gun, and attack-state logic from BasicEnemyAI.

Figure Captions and Explanations

Figure 1. Runtime data-flow diagram for one gameplay floor.

The data-flow diagram shows how external input enters the runtime, how the main gameplay processes transform it, and where persistent runtime data is stored. The player provides keyboard

and mouse input. PlayerController transforms that input into movement, interaction, and fire requests. Combat requests are handled by Gun and enemy target objects. Terminal, reward, and shop interactions are handled through the interactable pipeline. Progression data is owned by CybergrindArenaDirector and CybergrindRunState, while the HUD reads derived state from those systems and presents it to the player.

Figure 2. Structure chart for the runtime floor loop.

The structure chart uses module hierarchy plus control and data couples. Open circles represent data couples, which pass values such as input state, floor state, target state, and HUD values. Filled circles represent control couples, which trigger actions such as fire, interact, floor clear, or start transition. The loop marker indicates behaviour that repeats every frame during active play.

Figure 3. Main class relationships and OOP contracts.

The class diagram should emphasise inheritance, interface realisation, and key ownership relationships. It is intended to show that the project uses abstract base classes for shared interaction and generation roles, while interfaces are used for cross-system contracts such as damage and grapple behaviour.

Section 3.5 Replacement Data Dictionary

This version is more technical because it records type, owner, role, constraints, initial or default behaviour, and update trigger.

Name	Type	Owner/ Class	Purpose	Valid range or rule	Initial/ default state	Updated when	Example
health	float	PlayerController, BasicEnemyAI	Current hit points used by damage and death logic.	0 <= health <= maxHealth	Spawned at max health.	Damage, healing, death reset.	83.5
momentum	Vector3	PlayerController	Preserved horizontal movement velocity between chained actions.	Runtime vector; clamped indirectly by movement rules.	Vector3.zero on spawn/ reset.	Every frame during movement solve.	(12.4, 0.0, -3.1)
isGrounded	bool	PlayerController	Whether the player is on a stable surface.	true or false	false until first valid ground check.	Ground probe and controller movement.	true
dashCharges	int	PlayerController	Remaining dash count before recharge.	0..MaxDashCharges	Maximum on spawn and many resets.	Dash use, recharge, grounded refresh.	2

grappleRange	float	PlayerController	Maximum distance for a valid grapple anchor.	Positive scalar; clamped in tuning defaults.	Inspector configured.	Retuned or reconfigured.	46.0f
activeGrappleRopeLength	float	PlayerController	Current effective rope length after latch and reel logic.	>= grappleMinRopeLength	Set to full range on launch.	Grapple update while latched.	14.7f
grappleMassClass	enum	BasicEnemyAI, Target	Whether grapple physics pulls the target or the player.	Light or Heavy	Defined by object or enemy type.	Spawn/setup and some enemy-type changes.	Heavy
activePresetIndex	int	Gun	Currently equipped weapon preset index.	Must be a valid preset index.	Usually pistol at run start.	Weapon reward, shop refit, manual switch.	2
nextTimeToFire	float	Gun	Absolute time gate for next primary shot.	>= Time.time when cooling down	0 when ready.	Every shot fired.	128.42f
nextAltFireTime	float	Gun	Absolute time gate for next alternate fire.	>= Time.time when cooling down	0 when ready.	Ability activation.	131.90f
runDamageMultiplier	float	Gun	Run-level modifier applied to base weapon damage.	Positive scalar	1.0f at run start.	Shop overclock and run reset.	1.18f

agent	NavMeshAgent	BasicEnemyAI	Navigation component used by ground enemies.	Must exist for nav-driven enemies.	Cached in setup.	Referenced throughout AI update.	NavMeshAgent@Enemy_12
target	Transform	BasicEnemyAI	Current player target for facing and attack logic.	Nullable reference	Set during setup or discovery.	Target acquisition and refresh.	Player.transform
floor	int	CybergrindArenaDirector	Current floor index used for scaling and progression.	floor >= 1	1 at run start.	Exit completion or floor advance.	5
arenaMode	enum	CybergrindArenaGenerator	Floor-generation mode.	Declared enum member	Determined by director before generation.	Floor transition or debug override.	Combat
isSolved	bool	Terminal	Whether a terminal objective is complete.	true or false	false on spawn.	Puzzle completion and reset.	true
shopPurchaseUsed	bool	CybergrindRunState	Prevents more than one purchase on the same floor.	true or false	false when entering a new floor.	Successful shop purchase.	true
floorTimerRemaining	float	CybergrindArenaDirector	Remaining time before the current timed floor expires.	0 <= remaining <= duration	Set from floor-duration calculation.	Decrements each frame while active.	42.6f

encounterStartTime	float	CybergrindArenaDirector	Timestamp used for pacing logic such as delayed priority highlighting.	Absolute game time	Set when floor combat starts.	Once per encounter start.	351.2f

Section 5 Testing Improvements

The testing section is already stronger than the planning diagrams, but it still needs harder evidence. The main issues are weak metric precision and inconsistent formal structure. White-box tests should be converted into explicit test IDs so that the report reads like an engineering document rather than a reflective summary.

Test ID	Test type	Code path	Procedure	Expected result	Actual result
WB-01	White-box	Gun -> IDamageable.TakeDamage	Fire at player, enemy, and target dummy with debug values enabled.	All three receive damage through the same interface call path.	Pass
WB-02	White-box	Gun.ResetTransientAbilityStateForSwitch()	Switch weapon during active ability state.	Temporary state and cooldowns reset for the switched weapon.	Pass

Performance should also be described with a fixed benchmark table rather than loose wording such as around 150 to 200 FPS.

Scenario	Resolution	Avg FPS	1% low FPS	Observation
Combat floor, 6 to 8 enemies	1920x1080	____	____	Stable during play
Floor generation transition	1920x1080	____	____	Lowest frame spikes occur here
Boss floor	1920x1080	____	____	More stable than generation, less stable than shop

Figure files to insert into the main report are stored in docs/report_revision/assets as figure1_dfd.svg, figure2_structure_chart.svg, and figure3_class_diagram.svg.